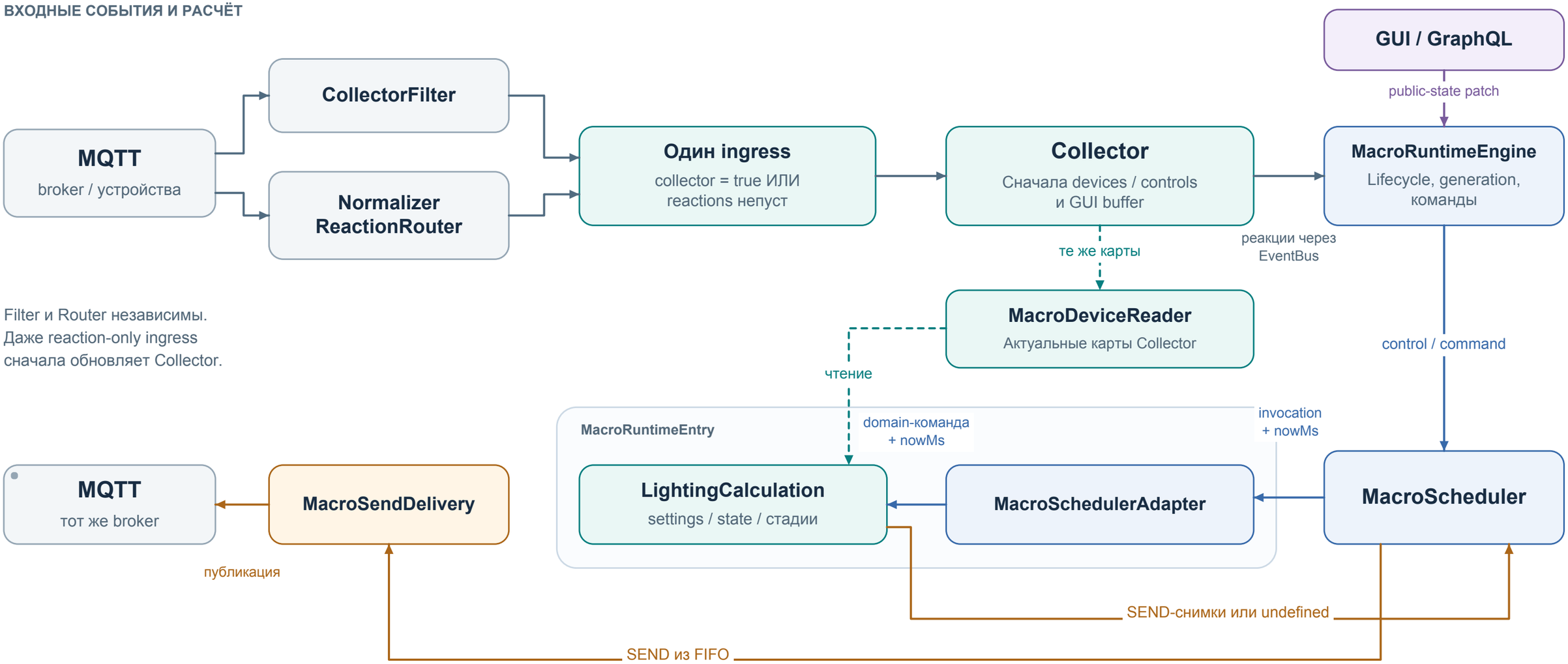
Общая цепочка: от MQTT до состояния и SEND

Новый runtime в production зарегистрирован для MacrosType.LIGHTING. Остальные входные типы обслуживает прежний MacrosEngine.

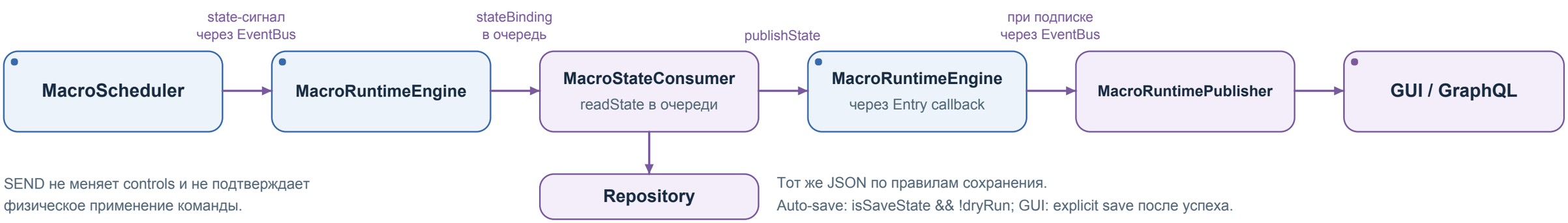
ВХОДНЫЕ СОБЫТИЯ И РАСЧЁТ



Filter и Router независимы.
Даже reaction-only ingress
сначала обновляет Collector.

ОЧЕРЕДЬ СОСТОЯНИЯ И ПУБЛИКАЦИЯ

Повторные узлы обозначают те же компоненты



SEND не меняет controls и не подтверждает
физическое применение команды.

Тот же JSON по правилам сохранения.
Auto-save: isSaveState && !dryRun; GUI: explicit save после успеха.

Внутри MacroScheduler: выбор готовой работы

Шесть классов работы. Курсор и использованная квота сохраняются между turns; пустые классы пропускаются.



FIFO задаёт порядок начала доставок, не завершения Promise.

Budget: 8 мс

Проверка после полного шага.
Начатый execute не прерывается.
TimerPort: wall clock + monotonic clock.

Background: общий круг

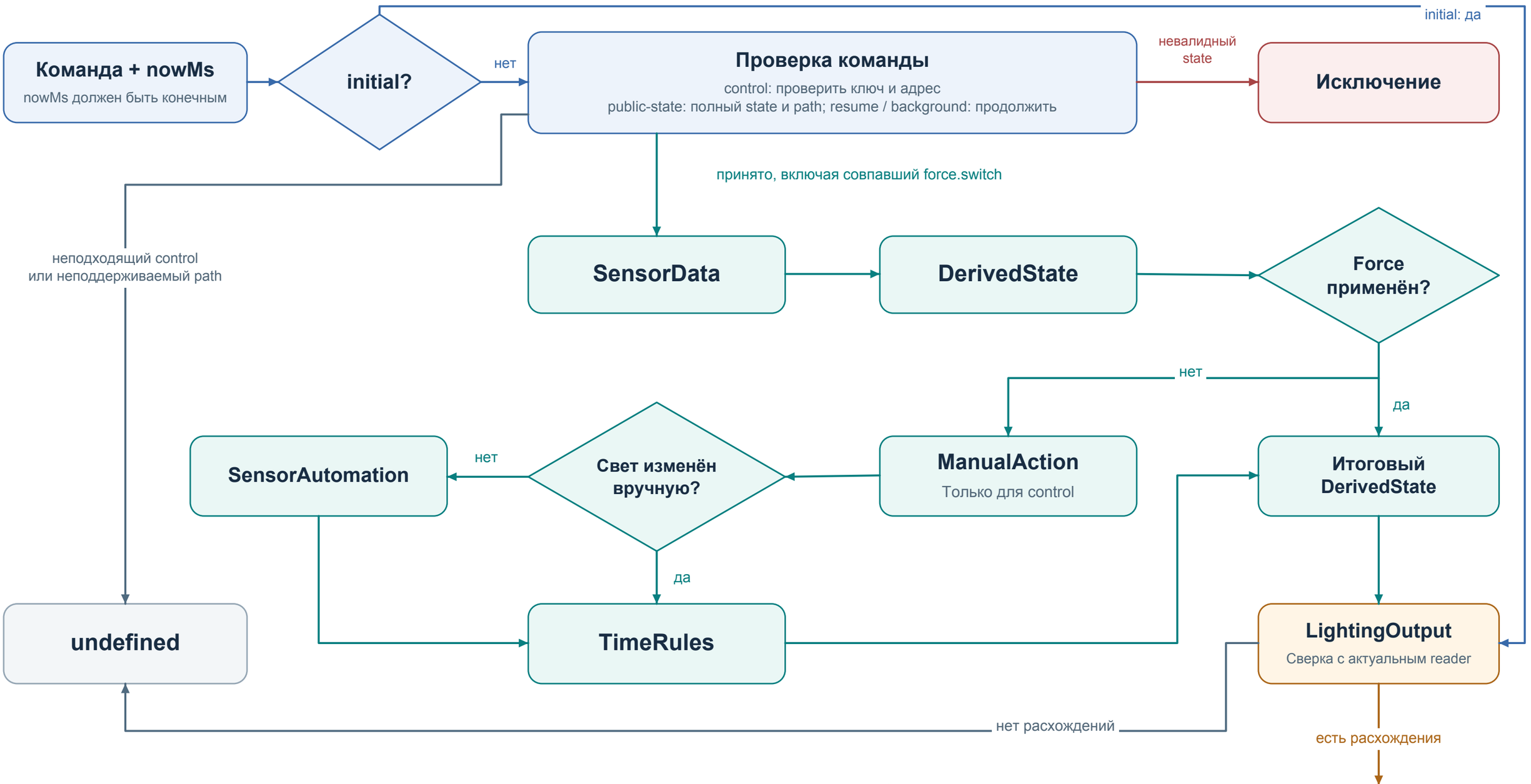
Следующий экземпляр готов через 200 мс.
Это не период каждого макроса.
У LIGHTING пустой schedule plan.

Ёмкость и ошибки

urgent + normal + GUI: controlCapacity.
У SEND отдельная ёмкость.
Overflow отклоняет новый элемент.
Throw не создаёт SEND / state-сигнал.

Внутри LightingCalculation: стадии и развилки

Один execute принимает команду и nowMs. Стадии изменяют state; единственный финальный Output формирует результат.



Важные условия

public-state принимает только path ['force', 'switch'].
Force пропускает ManualAction, SensorAutomation и TimeRules.
initial согласует выход с сохранённым state без business pipeline.

Неизвестная команда также вызывает исключение.